

Personal Projects

아이디어가 생기면 웹·모바일로 직접 만들어 보는 개인 GitHub입니다.
프론트엔드 중심 풀스택으로 기획 → 구현 → 배포까지 해보고,
AI 협업·CI/CD·자동화 도구 같은 워크플로우도 함께 시도합니다.
괜찮아진 것은 오픈소스로 남깁니다.

<https://github.com/dayainow>

요약

주요 영역	프론트엔드 · 풀스택 · 프로덕트 (웹/앱, 어드민·커머스 UI)
제품	모바일 앱 2종 배포 · 웹 서비스 MVP~운영 · 3D 인터랙션 프로토타입
오픈소스	CI/CD 자동화, Figma↔코드 동기화, 모바일 검증, 컴포넌트 격리 개발 등 14+ 재사용 패키지
관심사	TypeScript/React, 복잡한 UI-상태 설계, AI를 검증 가능한 워크플로로 쓰는 것

사용자에게 닿는 제품을 끝까지 만들고, 검증된 패턴은 도구와 문서로 남깁니다.

대표 프로젝트

웹 서비스

프로젝트	한 줄 소개	기술
OlaLab	100+ AI 도구 큐레이션·커뮤니티 플랫폼	Next.js · NestJS · Prisma · Supabase
HarnessHub	AI 에이전트·하네스 발견·평가·설치 카탈로그 (3D UI, CLI)	Next.js · NestJS · R3F · Redis

모바일 · 인터랙션

프로젝트	한 줄 소개	기술	링크
오늘의 도서관	전국 공공도서관 지금 열린 곳 검색·즐거찾기 ·길찾기	Expo · TypeScript · Vercel API	today-library-sigma.vercel.app
0원의품격	0원 문화행사·전시 정보 앱	Expo · TypeScript · Vercel API	zero-won-poomgyeok.vercel.app
메모빌	감정 일기 → 3D 마을 성장 인터랙션	Expo · Three.js · R3F	prototype-web-eosin.vercel.app

핵심 역량

Frontend — TypeScript · React · Next.js · Tailwind · Zustand · TanStack Query
복잡한 폼·권한·전역 상태 · 반응형 UI · Storybook/격리 개발 패턴

Mobile — Expo · React Native · EAS · 위치·검색·오프라인 즐겨찾기

Backend — NestJS · Node.js · Prisma · Supabase · Vercel Serverless · Redis

Delivery — GitHub Actions(CI/CD) · Vercel 배포 · SEO/OG · Rate Limiting

AI Workflow — Cursor · MCP · Figma MCP · 역할 분리 에이전트 · Skill/Rule 표준화

영역	실무에서 하는 일
프론트엔드	B2C/B2B UI 구현, 디자인 시스템 연동, 성능·접근성 개선
풀스택	API 설계·연동, DB 스키마, 인증·권한, 배포 파이프라인
AI 활용	코드 생성기가 아닌 검증 가능한 워크플로로 운영 (역할·게이트·재현)
도구화	반복 작업을 npm 패키지·GHA 워크플로·Cursor Skill로 추출

오픈소스 — 개발 자동화 & 워크플로

제품 개발 중 만들어 검증한 도구입니다. 이식 가능한 패키지·템플릿 형태로 공개합니다.

배포 · 품질

- **CI/CD Harness** — build → test → deploy 표준화, GHA Summary·HTML 리포트·Slack/Discord 알림
- **Frontend Collab Kit** — 팀 단위 ESLint/Prettier/Husky + 컴포넌트 스캐폴딩 CLI

AI 협업 · 디자인 연동

- **Figma Publish Harness** — Figma ↔ Next.js 양방향 퍼블·동기화
- **Role-Based AI Harness** — planner → 설계 → 구현 → QA 역할·핸드오프 방법론
- **3-Layer Harness** — Hooks · 공유 지침 · 전문 에이전트 템플릿
- **Web Performance Audit Skill** — Lighthouse → 병목 진단 → 개선안 → 재검증

프론트엔드 개발 · 검증

- **Component Harness** — UI 격리 샌드박스, spec 기반 deterministic 검증
- **Auth Permission Harness** — 모노레포 권한·기관 Mock
- **Audio Player Harness** — 전역 플레이어 상태 주입·MP3/HLS/에러 시나리오
- **WebView Bridge Harness** — Flutter WebView 브릿지 브라우저 Mock
- **Web Vitals RUM Harness** — 실사용자 Web Vitals 수집 → Lighthouse 분석 연결

모바일 · 앱 개발 검증

- **Expo Release Harness** — Expo/EAS 빌드 · OTA 업데이트 · 스토어 배포 전 체크리스트 자동화
- **Mobile Permission Harness** — 위치·카메라·알림·사진 접근 권한 Mock
- **Deep Link Harness** — 앱 딥링크·Universal Link·App Link 라우팅 검증
- **Offline Sync Harness** — 오프라인 저장·재시도 큐·네트워크 복구 시 동기화 시나리오 검증

자동화 구성 방식

1. AI 규칙 — Cursor Skill · Rule · Prompt (협업 순서·품질 게이트)
2. 로컬 품질 — Husky · ESLint · Prettier (커밋 시)
3. UI 검증 — Component Harness · Storybook (개발 중)
4. CI/CD — GitHub Actions · CI/CD Harness (push 후 build/test/deploy)

기술 스택

- **Language** — TypeScript (주력), JavaScript
- **Frontend** — React, Next.js App Router, Tailwind CSS, Zustand, TanStack Query
- **Mobile** — Expo, React Native, EAS Build
- **Backend** — NestJS, Node.js, Prisma, Supabase, Vercel Serverless
- **3D / UX** — Three.js, React Three Fiber, Framer Motion
- **Data** — 공공데이터 API, CDN 캐시, cron, seed fallback
- **Infra** — Vercel, GitHub Actions, Redis, SEO/OG, JSON-LD
- **Dev / AI** — Cursor, Claude, MCP, Figma MCP, Lighthouse, web-vitals

일하는 방식

아이디어 → MVP → 실사용 검증 → 패턴 추출 → 오픈소스·다음 제품에 재사용

- **End-to-end** — 기획·UI·API·배포·출시 준비까지 직접 수행
- **프로토타입 우선** — 동작하는 제품 루프를 먼저 만들고 구조를 다듬음
- **품질 게이트** — 린트·테스트·CI를 코드와 함께 유지
- **UX** — 정보는 충실하되 화면은 과하지 않게

현재 관심

메모빌 — 감정 일기가 3D 마을로 시각화되는 인터랙티브 다이어리

DiaryEntry → DiaryMemory → VillageEffect[] → VillageSnapshot → Three.js Village

작은 것을 만들고, 쓸 만해지면 더 단단하게 만듭니다.